Práctico 05 — Cámara en la escena

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

18 de septiembre de 2026



Roadmap de la clase

Antecedentes

- Unidad VII — visualización 3D
- Un avión armado con primitivas
- Una matriz de modelo por pieza

Hoy

- **Cámara** en la escena
- **Proyección** de la visualización
- Equivalencia entre el teórico y OpenGL

Conectado a

- Unidad VIII — caras ocultas
- Unidad IX — iluminación
- El proyecto: sistema de cámaras

Las cajas negras de hoy

- Se cierra la constante `uAjuste` en el vertex shader.

Lo que dice el código del Práctico 03

assets/shaders/solid.vs

```
// CAJA NEGRA, vence el 18-sep.
// Matriz de vista y proyeccion
// combinadas. Se calcula en CPU
// y se pasa como uniform.
uniform mat4 uAjuste;

gl_Position = uAjuste * uModel
              * vec4(aPos, 1.0);
```

src/main.cpp

```
const glm::mat4 m_ajuste =
  glm::scale(glm::mat4(1.0f),
    glm::vec3(600.0f/800.0f,
              1.0f, -1.0f));
```

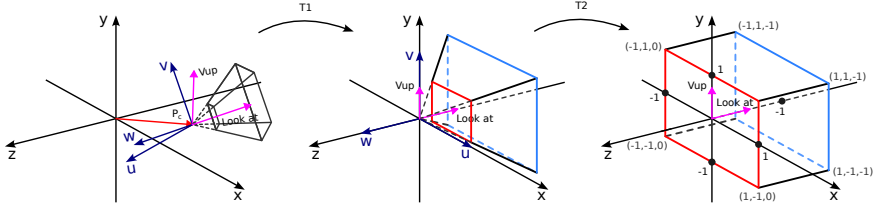
Un factor de escala por eje.

Entonces, ¿qué es `uAjuste`?

No es una proyección en perspectiva. Es un escalado: corrige la relación de aspecto de la ventana e invierte el sentido del eje z.

Hasta hoy no hubo perspectiva en pantalla. Lo que se visualiza es una proyección paralela, con la cámara fija en el origen.

Teoría: transformación de normalización



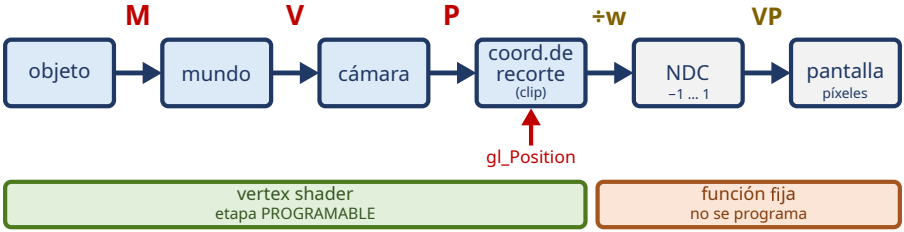
La **transformación de normalización**: llevar una visualización arbitraria a la canónica.

$$\overline{q}\,' = D \cdot (S_2)_{xyz} \cdot S_{xy} \cdot M^T \cdot T(-\overline{P_c}) \cdot \overline{q}$$

$T(-\overline{P_c})$ lleva la cámara al origen $\quad M^T$ alinea (u, v, w) con (x, y, z) $\quad S_{xy}$, $(S_2)_{xyz}$ ajustan el volumen

D deforma el frustum

Aplicación: la transformación en OpenGL



M matriz de modelo — uModel
V matriz de vista — glm::lookAt(eye, center, up)
P matriz de proyección — glm::perspective(...)

÷w división por w (homogeneización)
VP transf. de viewport — glViewport
el recorte se realizá acá

Un vértice no se digitaliza directamente: **cambia de sistema de referencia cinco veces** antes de convertirse en píxeles.

Cómo es la equivalencia

Teórico	OpenGL / glm	CGP&P - Cap. 13[1]
Unidad V y VI	**Model** $\rightarrow$ `uModel`	`CMTM`
$M^T \cdot T(-\overline{P_c})$	**View** $\rightarrow$ `glm::lookAt()`	N (ec. **13.20**)
—	*"modelview matrix"* $= V \cdot M$	$N \cdot$ `CMTM`
$D \cdot (S_2)_{xyz} \cdot S_{xy}$	**Projection** $\rightarrow$ `glm::perspective()`	resto de $M_{pp} M_{per}$
transf. completa	**MVP** $= P \cdot V \cdot M$	`CTM` (ec. **13.19**)

[1] Computer Graphics: Principles and Practice - Hughes, Foley, van Dam, et.al.

Cuáles son las diferencias

¿Por qué la convención de uso práctico es dividir en tres el proceso de transformación?

Matriz	Cambia...
P	al redimensionar (casi nunca)
V	una vez por cuadro
M	una vez por objeto

La división está en la reutilización. No es una imposición de la API.

¿Por qué la matriz de proyección de OpenGL no coincide con D?

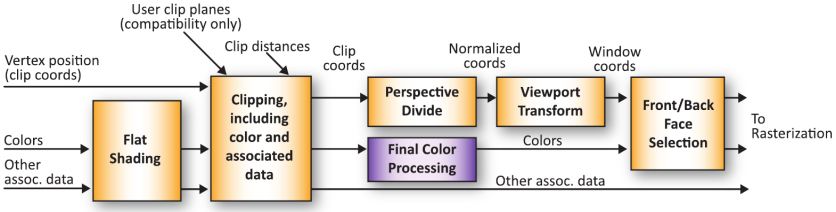
Porque `perspective` **no es** D: es $D \cdot (S_2)_{xyz} \cdot S_{xy}$, **los tres factores de proyección** —la fila *Projection* del cuadro anterior—.

Con los tres, la coincidencia es exacta salvo en el **volumen canónico**:

Teórico	$-1 \leq x, y \leq 1$	;	$0 \leq z \leq -1$
OpenGL	$-1 \leq x, y \leq 1$	;	$-1 \leq z \leq 1$

Distinto recorrido en z, distinta fila de z. **Las demás filas de la matriz son las mismas.**

OpenGL: *Under the hood*



OpenGL 4.6 API Reference Guide, p. 8 — © 2017 Khronos Group

Cuál es el *paso 3* del teórico: proyectar y recortar usando el volumen canónico

En OpenGL esto corresponde a: **recorte** (*clipping*) y **división por** *w perspective divide*.

Finalmente la transformación a coordenadas del dispositivo se hace en **viewport**.

Los tres son función fija. De la etapa programable del pipeline sale `gl_Position` en *clip coords*.

La cámara no se mueve en realidad

El teórico enuncia, sobre las transformaciones de normalización:

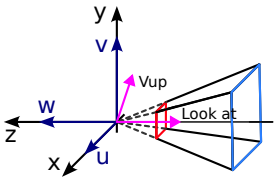
"Se aplica a todos los vértices de los objetos para preservar las distancias entre cámara y escena"

La cámara **nunca** se mueve. Se mueve la escena, para que la cámara quede en la posición canónica.

Por eso las dos primeras transformaciones que se aplican son **inversas**:

$$V = M^T \cdot T(-\overline{P_c})$$

$T(-\overline{P_c})$: traslación *inversa*. $M^T = M^{-1}$: rotación *inversa*.



Cámara en la posición canónica

La **matriz de vista** corresponde a esta composición de transformaciones.

Importante

La matriz de vista no ubica la cámara: es la *inversa* de la pose de la cámara.

Cálculo de la matriz de vista: implementación propia

$$V = M^T \cdot T(-\overline{P_c}) = \begin{bmatrix} u_x & u_y & u_z & -\overline{u} \cdot \overline{P_c} \\ v_x & v_y & v_z & -\overline{v} \cdot \overline{P_c} \\ w_x & w_y & w_z & -\overline{w} \cdot \overline{P_c} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Las **filas** son la base (u, v, w): por eso la rotación entra transpuesta, M^T, y no M.

La traslación inversa

$$T(-\overline{P_c}) = \begin{bmatrix} 1 & 0 & 0 & -P_{cx} \\ 0 & 1 & 0 & -P_{cy} \\ 0 & 0 & 1 & -P_{cz} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La base ortonormal, a partir del punto observado $\overline{P_{ref}}$ (*Look at*) y del vector $\overline{Up}$:

$$\overline{w} = \frac{\overline{P_c} - \overline{P_{ref}}}{\|\overline{P_c} - \overline{P_{ref}}\|} \qquad \overline{u} = \frac{\overline{Up} \times \overline{w}}{\|\overline{Up} \times \overline{w}\|} \qquad \overline{v} = \overline{w} \times \overline{u}$$

$\overline{w}$ apunta en sentido contrario a *Look at*: la cámara mira hacia $-z$.

$\overline{Up}$ no necesita ser perpendicular a *Look at*, pero **no puede ser colineal**: el producto vectorial se anula.

La misma matriz, con `glm::lookAt`

```
glm::mat4 view = glm::lookAt(eye,      // donde esta la camara  (Pc)
                             center,   // punto observado       (Look at)
                             up);      // referencia de vertical (Up)
```

Devuelve **exactamente** $M^T \cdot T(-\overline{P_c})$: la matriz de la filmina anterior, **ya invertida**.

Para ubicar la cámara **10 unidades hacia atrás** se escribe:

Incorrecto

$V = T(0, 0, +10)$

Es la **pose** de la cámara.

La escena se desplaza hacia el **mismo lado** que la cámara.

Correcto

$V = T(0, 0, -10)$

Es la **inversa** de la pose.

`lookAt` la devuelve así.

Cómo se reconoce

La escena se desplaza en sentido contrario al esperado, y exactamente en sentido contrario: si al mover la cámara hacia la izquierda la escena también se va hacia la izquierda, el error es éste.

Proyección paralela: `glm::ortho`

La normalización de una proyección paralela es la misma cadena del teórico **sin la transformación** D: el volumen de visualización ya es un prisma(no hay que deformar una pirámide).

$$\overline{q}' = (S_2)_{xyz} \cdot S_{xy} \cdot M^T \cdot T(-\overline{P_c}) \cdot \overline{q}$$

```
glm::mat4 proj = glm::ortho(left, right,     // limites en x
                            bottom, top,     // limites en y
                            near, far);      // planos de recorte
```

La cuarta fila queda en $(0, 0, 0, 1)$: **no hay división por** w (homogeniezación), y por eso las proporciones se conservan a cualquier distancia.

Por qué la visualización funcionaba hasta ahora

`scale(600/800, 1, -1)` $\equiv$ `ortho(-4/3, 4/3, -1, 1, -1, 1)`

`uAjuste` **simulaba una proyección ortográfica exacta**

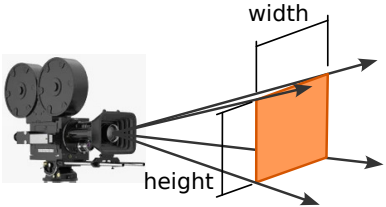
El ajuste de aspecto: de dónde sale el 600/800

La transformación de escalado en x e y es:

$$S_{xy} = \begin{bmatrix} \cot\frac{\theta_w}{2} & 0 & 0 & 0 \\ 0 & \cot\frac{\theta_h}{2} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Los dos ángulos están vinculados por la relación de aspecto:

$$\tan\frac{\theta_w}{2} = \mathrm{AR} \cdot \tan\frac{\theta_h}{2} \quad \Longrightarrow \quad \cot\frac{\theta_w}{2} = \frac{\cot\frac{\theta_h}{2}}{\mathrm{AR}}$$



Relación de aspecto y ángulos de visión

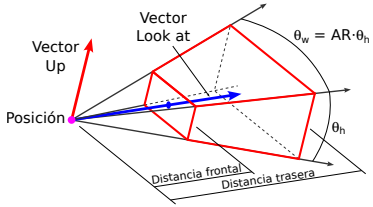
Qué le faltaba a uAjuste para ser una perspectiva

El 600/800 era 1/AR, sólo el ajuste de aspecto. Faltaban el $\cot\frac{\theta_h}{2}$ —por eso no había campo visual— y **la cuarta fila, la que hace** $w = -z$: sin ella no hay división por w, y sin división por w no hay perspectiva.

Proyección en perspectiva: `glm::perspective`

```
glm::mat4 proj = glm::perspective(
        glm::radians(45.0f),   // fovy
        (float)width/height,   // aspect
        0.1f,                  // near
        100.0f);               // far
```

glm	Teórico Unidad VII
`fovy`	θ_h — ángulo de visión vertical
`aspect`	se usa para derivar θ_w
`near`	plano de recorte frontal
`far`	plano de recorte trasero



Parámetros de visualización — perspectiva

Radianes, no grados

`fovy` va en **radianes**. Si se pasa 45 en vez de `glm::radians(45.0f)`, **no hay error**: hay pantalla negra.

Implicancias en la selección de `near` y `far`

Definen la ubicación de dos de los planos de recorte. Pero además **impactan en la precisión** con la que se guarda la profundidad de los triángulos a dibujar.

- Puntos **equiespaciados** en el frustum **no** quedan equiespaciados después de proyectar: se **distribuyen con mayor densidad hacia el plano trasero**.
- Cuanto menor sea el valor de `near`, mayor es la densidad hacia "atrás".
- Si dos superficies tiene valores de profundidad muy cercanos, **el sistema no puede decidir cuál ocluye a cuál** (fenómeno de *z-fighting*).

Recomendación

"Colocar el plano delantero (near) tan lejos del ojo como se pueda, sin dejar de eliminar lo que hace falta visualizar." **(Ver §13.6 en Computer Graphics: Principles and Practice de Hughes, Foley, van Dam, et.al.)**

La distancia `near` importa mucho más que la distancia `far`.

Por ahora caja negra: por qué esto puede alterar la imagen se ve en la Unidad VIII, z-buffer. Usar la recomendación.

GLFW: la ventana, el *framebuffer* y los eventos

GLFW administra la ventana y el contexto de OpenGL. El tamaño de esa ventana define el tamaño del **framebuffer** —la grilla de píxeles sobre la que dibuja OpenGL—, y de ahí sale la relación de aspecto.

Y ofrece **dos mecanismos** para enterarse de lo que hace el usuario:

Naturaleza	Mecanismo	Ejemplo de hoy
control **continuo** (estado)	*polling*	mover la cámara con el mouse
comando **discreto** (evento)	*callback*	redimensionar la ventana

Un *callback* es una función propia que se **registra** en la librería, y que la librería llama cuando el evento ocurre:

```
glfwSetFramebufferSizeCallback(window, framebuffer_size_callback);
```

Redimensionar la ventana

aspect depende del tamaño del *framebuffer*. Si la ventana cambia y la matriz de proyección no, **la escena se deforma**.

```
void framebuffer_size_callback(GLFWwindow* window, int width, int height)
{
    glViewport(0, 0, width, height);          // adonde se dibuja

    if (height > 0) {                         // al minimizar puede ser 0
        g_proyeccion = glm::perspective(glm::radians(45.0f),
                                        (float)width / height,
                                        0.1f, 100.0f);
    }
}
```

Son dos cosas distintas:

- `glViewport` → el rectángulo de píxeles del *framebuffer*.
- P → la forma del frustum.

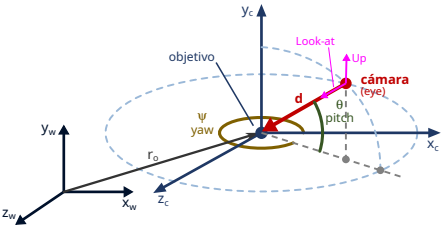
Y las dos hacen falta:

- Sin `glViewport`, se dibuja en el rectángulo viejo.
- Sin recalcular P, la escena se estira.

La cámara orbit: qué decide el usuario

La cámara se describe con **tres números** alrededor de un punto objetivo: dos ángulos y una distancia.

El usuario no fija esos tres números: pide **cambios** sobre ellos.



yaw — ang. longitud sobre la órbita
pitch — ang. latitud (limitado)

d — distancia al objetivo
la cámara siempre mira al objetivo

```
struct CameraCommand {   // deltas
    float yaw_delta   = 0.0f;
    float pitch_delta = 0.0f;
    float dist_delta  = 0.0f;
};
```

De esféricas a cartesianas sale la posición **relativa al objetivo**, y de ahí los tres argumentos de `lookAt`:

$$\texttt{eye} = \texttt{objetivo} + d\,(\cos p \cos y,\ \sin p,\ \cos p \sin y)$$

$$\texttt{center} = \texttt{objetivo} \qquad \texttt{up} = (0, 1, 0)$$

Por qué deltas y no valores absolutos

Quien lee el mouse **no tiene por qué conocer** el estado de la órbita ni sus límites. Acumular y acotar es responsabilidad de la cámara.

CameraSystem: de la pose al par de matrices

```
struct CameraData {            // lo unico que sale del modulo
    glm::mat4 view;
    glm::mat4 projection;
};
class CameraSystem {
public:
    CameraSystem(int width, int height);       // arma la proyeccion inicial
    void update(glm::vec3 pos_avion,           // el objetivo
                glm::vec3 angulos_avion,       // HOY no se usa
                const CameraCommand& cmd);     // una vez por cuadro
    void set_viewport(int width, int height);  // desde el callback de resize
    const CameraData& data() const;
};
```

Qué **no** hace

- **No lee el mouse**. Recibe *deltas* ya interpretados.
- **No dibuja**. No conoce a OpenGL más allá de `glm`.
- **No sabe qué hay en la escena**. Recibe un punto al que mirar y nada más.

`InputHandler`: hoy, sólo el mouse

Falta quién produzca esos *deltas*. Mover la cámara es un **control continuo**: lo que interesa en cada cuadro es el **estado** del mouse, así que se lo consulta por *polling*.

```
void InputHandler::update(GLFWwindow* window, float dt)
{
    double x, y;
    glfwGetCursorPos(window, &x, &y);        // POLLING: se lee el estado
    // delta = posicion actual - posicion anterior
}
```

El criterio

La elección entre *polling* y *callback* **no es de estilo ni de rendimiento: es semántica.** Se decide por la naturaleza de la acción —estado o evento—, no por el dispositivo.

Qué NO hace: no mueve la cámara y no conoce a `CameraSystem`. Traduce píxeles a radianes y entrega un `CameraCommand`.

Práctico 05 — lo de la semana

1. **Partir** `uAjuste` en `uView` y `uProjection`. Los tres uniforms, en el orden $P \cdot V \cdot M$.
2. **Una cámara orbit**: yaw, pitch y distancia alrededor de un punto. De ahí salen `eye`, `center` y `up` para `lookAt`.
3. **Recalcular la proyección al redimensionar**, por callback.

El requisito 5 se reparte

El kickoff decía *18-sep* → *requisito 5*. Se corrige: **hoy la cámara, el 25-sep las tres vistas.**
Con el avión quieto, tres vistas distintas son **tres cámaras fijas**: no hay forma de ver si siguen al avión. Se piden cuando se pueda verificar.

Sobre la firma que se ofrece

El enunciado ofrece una **firma** para `CameraSystem`. Es un punto de partida, no un molde: toda variante tiene que poder justificarse.

El enunciado completo queda en el aula.

La escena ya tiene cámara.

Y gira a **distinta velocidad**
en cada máquina.

¿Por qué?